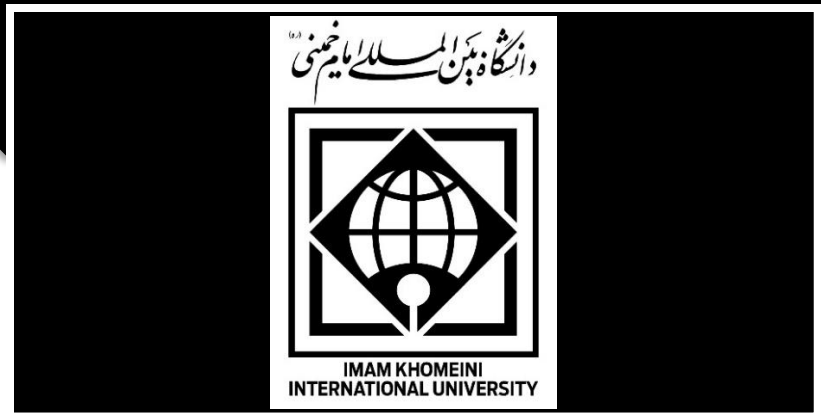




پروژه پیاده‌سازی Pipeline استاندارد MLOps برای پیش‌بینی ریزش مشتری

هدف این پروژه، طراحی و پیاده‌سازی یک Pipeline استاندارد MLOps برای پیش‌بینی ریزش مشتریان با استفاده از دیتاست ماشین آموزش و ارزیابی می‌شوند. تمامی آزمایش‌ها، پارامترها و نتایج در MLflow ثبت خواهند شد تا قابلیت ردیابی و مقایسه مدل‌ها فراهم شود. در نهایت، مدل منتخب در قالب یک سرویس قابل استقرار با استفاده از Docker ارائه می‌شود تا فرآیند آموزش، مدیریت و استقرار مدل به‌صورت استاندارد، قابل تکرار و عملیاتی انجام گیرد.

مراحل انجام پروژه

1. دیتاست

در این پروژه از دیتاست Telco customer churn: IBM dataset استفاده می‌کنیم که شامل اطلاعات حدود ۷ هزار مشتری یک شرکت مخابراتی است. هدف، پیش‌بینی ریزش مشتری (Churn) است:

- Churn Value = 1 ← مشتری رفته
- Churn Value = 0 ← مشتری مانده

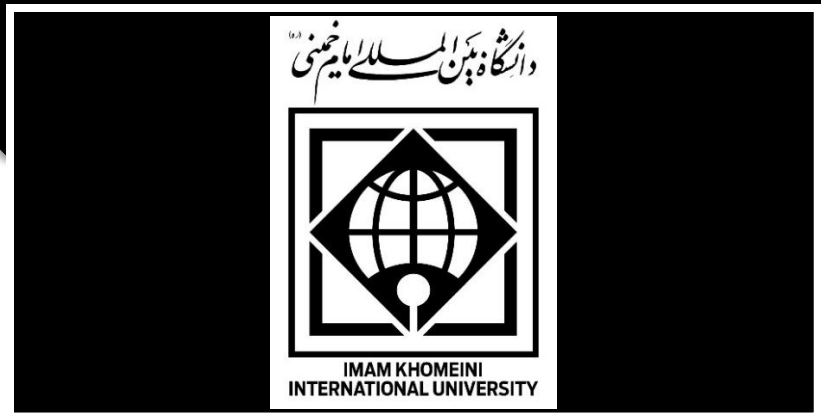
مدیریت نسخه داده (Data Versioning)

مرحله ۱: نسخه خام (v1):

- استفاده از داده خام بدون تغییر

مرحله ۲: پاک‌سازی داده (v2)

- حذف ستون‌های غیرضروری
- رفع missing values
- تبدیل داده‌های categorical به عددی (Encoding)



مرحله ۳: Feature Engineering (v3)

- ساخت ویژگی‌های جدید
- نرمال‌سازی داده‌های عددی

نکته مهم:

برای هر نسخه از دیتاست، فایل‌های مربوطه در پوشه‌ای مجزا (مانند data/v1, data/v2 و data/v3) ذخیره شده و تغییرات آن‌ها از طریق Git ثبت و نگهداری می‌شود. عملکرد مدل‌ها روی هر نسخه از داده به صورت مستقل ارزیابی و مقایسه خواهد شد. همچنین در هر آزمایش، اطلاعات مربوط به نسخه دیتاست، مدل، پارامترها و معیارهای ارزیابی در MLflow ثبت می‌شوند تا امکان مقایسه و بازتولید نتایج فراهم باشد. در صورت نیاز، می‌توان نسخه‌های جدیدی از داده ایجاد و با اعمال بهبودهای بیشتر، کیفیت داده و عملکرد مدل را ارتقا داد.

2. آموزش مدل و ارزیابی

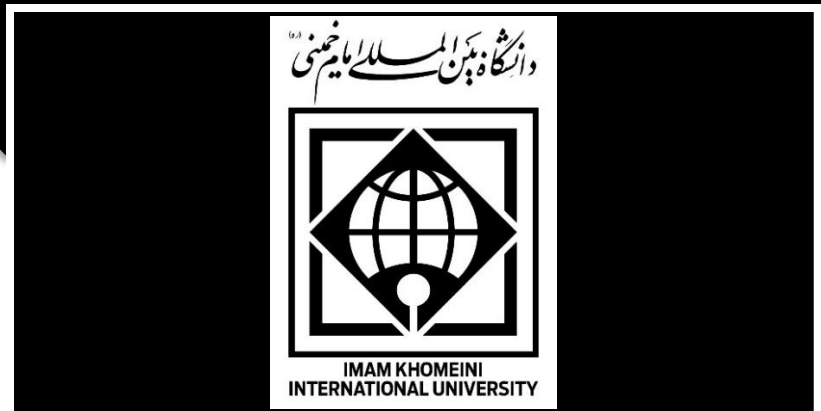
در این مرحله، مدل‌های مختلف Machine Learning روی داده‌های آماده‌شده آموزش داده می‌شوند و عملکرد آن‌ها ارزیابی می‌گردد.

برای جلوگیری از overfitting و داشتن نتایج پایدار:

- از Cross Validation (K-Fold) استفاده می‌شود
- داده‌ها به صورت train / validation / test تقسیم می‌شوند
- تمام مراحل با seed ثابت اجرا می‌شوند تا نتایج قابل تکرار باشند

مدل‌های مورد استفاده

- Logistic Regression
- Random Forest
- XGBoost
- CatBoost



برای هر مدل، معیارهای زیر ثبت و مقایسه می‌شوند:

- Accuracy
- Precision
- Recall
- F1-score
- ROC-AUC

در هر اجرا:

- نام مدل
- پارامترها (hyperparameters)
- نسخه دیتاست
- متریک‌ها
- seed استفاده شده
- Confusion Matrix

در MLflow ذخیره می‌شود تا امکان مقایسه و بازتولید کامل نتایج وجود داشته باشد.

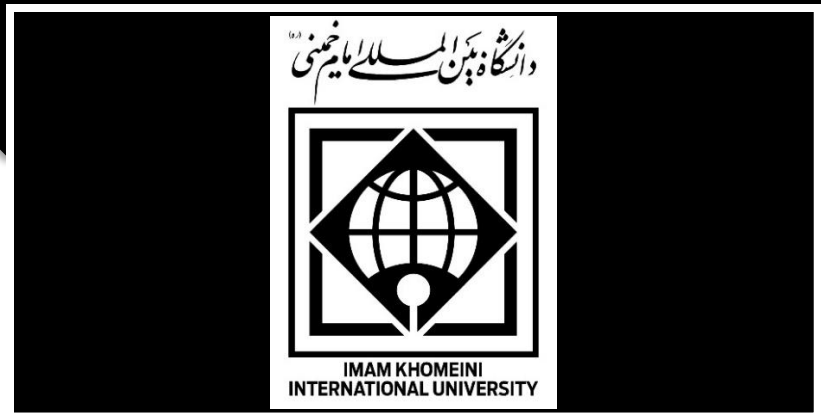
نکته مهم درباره روند آموزش و ارزیابی

برای هر نسخه از دیتاست:

- چندین بار آموزش مدل با استفاده از Cross Validation روی داده Train انجام می‌شود تا عملکرد مدل پایدار و قابل اعتماد باشد.

- از داده Validation برای انتخاب مدل نهایی و تنظیمات (hyperparameters) استفاده می‌شود.
- در نهایت، تنها یک بار ارزیابی روی داده Test انجام می‌شود تا عملکرد واقعی مدل در هر نسخه از دیتاست گزارش شود.

این فرآیند برای هر نسخه از دیتاست به صورت مستقل تکرار می‌شود؛ یعنی با تغییر نسخه داده (مثلاً از v1 به v2)، دوباره همین چرخه شامل Cross Validation، انتخاب مدل و ارزیابی نهایی روی Test اجرا می‌گردد.



3. ساخت Pipeline قابل تکرار (MLOps Pipeline)

در این مرحله، کل فرآیند به یک Pipeline استاندارد و قابل تکرار تبدیل می‌شود تا تمامی مراحل از بارگذاری داده تا ثبت نتایج به صورت خودکار اجرا شوند.

مراحل pipeline:

- بارگذاری نسخه موردنظر دیتاست از پوشه data
- preprocessing
- feature engineering
- training
- evaluation
- MLflow در logging

روش استاندارد اتوماتیک‌سازی

یک فایل اصلی برای اجرای Pipeline ایجاد می‌شود؛ برای مثال:

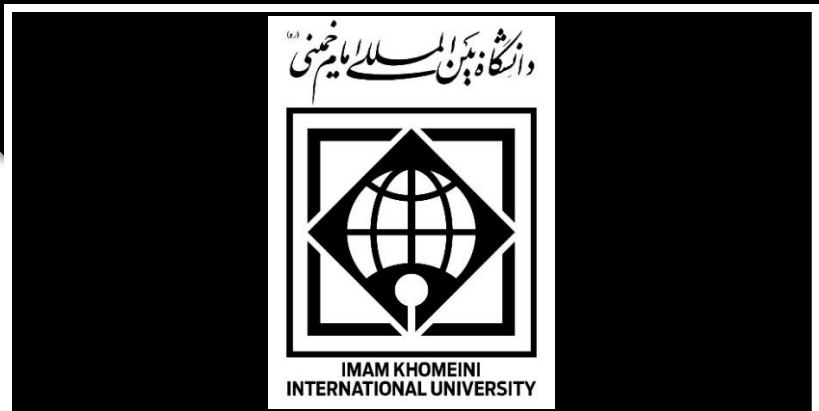
run_pipeline.py

محتوای فایل:

```
def main():  
    load_data()  
    preprocess()  
    feature_engineering()  
    train_models()  
    evaluate()  
    log_to_mlflow()  
  
if __name__ == "__main__":  
    main()
```

4. استقرار مدل (Model Deployment) با MLflow و Docker

در این مرحله، بهترین مدل آموزش‌دیده و ثبت‌شده در MLflow به صورت یک Docker Image بسته‌بندی و مستقر می‌شود تا امکان انجام پیش‌بینی (Inference) فراهم گردد.



در این روش، مدل نهایی مربوط به آخرین نسخه دیتاست از MLflow دریافت شده و با استفاده از قابلیت‌های MLflow در قالب Docker اجرا می‌شود.

در نتیجه:

- بهترین مدل در MLflow ثبت و نگهداری می‌شود.
- Docker Image از مدل ثبت شده در MLflow ساخته می‌شود.
- مدل مستقر شده، آخرین مدل آموزش دیده روی آخرین نسخه دیتاست است.

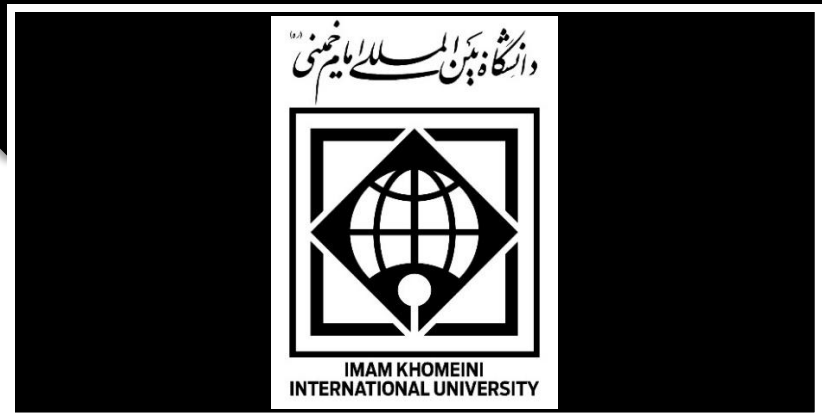
5. انتظارات پروژه

در پایان پروژه، موارد زیر باید رعایت و ارائه شوند:

- ایجاد یک Git Repository برای پروژه و ثبت تمامی تغییرات با Commit های منظم و دارای پیام مناسب (Commit Message) و ارسال آن به مخزن راه دور (Push).
- پیاده سازی پروژه مطابق با ساختار پوشه بندی زیر:

```
project/
├── data/
│   ├── v1/
│   ├── v2/
│   └── v3/
├── src/
│   ├── data_loader.py
│   ├── preprocessing.py
│   ├── features.py
│   ├── train.py
│   ├── evaluate.py
│   └── mlflow_utils.py
├── run_pipeline.py
└── README.md
```

- ثبت تمامی آزمایش‌ها، مدل‌ها و نتایج در MLflow.
- استقرار مدل نهایی با استفاده از Docker.



- تهیه مستندات مناسب پروژه (README.md) شامل معرفی پروژه، نحوه نصب، اجرای Pipeline، آموزش مدل، استقرار و نحوه استفاده از سرویس.
- رعایت اصول کدنویسی تمیز (Clean Code) شامل نام‌گذاری مناسب فایل‌ها، توابع و متغیرها و ماژولار بودن کد.
- تهیه گزارش مناسب پروژه (Documentation) شامل مراحل پیش‌پردازش و Feature Engineering، نسخه‌های مختلف دیتاست، مدل‌های استفاده‌شده، پارامترها، نتایج و مقایسه مدل‌ها، تصاویر خروجی MLflow، تصاویر گیت‌هاب و Commit‌های آن و نتیجه‌گیری